

磁盘取数据要用一段相对较长的时间(数量级为几十毫秒),所以内核执行从进程 A 到进程 B 的上下文切换,而不是在这个间歇时间内等待,什么都不做。注意在切换之前,内核正代表进程 A 在用户模式下执行指令(即没有单独的内核进程)。在切换的第一部分中,内核代表进程 A 在内核模式下执行指令。然后在某一时刻,它开始代表进程 B(仍然是内核模式下)执行指令。在切换之后,内核代表进程 B 在用户模式下执行指令。

随后,进程 B 在用户模式下运行一会儿,直到磁盘发出一个中断信号,表示数据已经从磁盘传送到了内存。内核判定进程 B 已经运行了足够长的时间,就执行一个从进程 B 到进程 A 的上下文切换,将控制返回给进程 A 中紧随在系统调用 read 之后的那条指令。进程 A 继续运行,直到下一次异常发生,依此类推。

8.3 系统调用错误处理

当 Unix 系统级函数遇到错误时,它们通常会返回 -1,并设置全局整数变量 `errno` 来表示什么出错了。程序员应该总是检查错误,但是不幸的是,许多人都忽略了错误检查,因为它使代码变得臃肿,而且难以读懂。比如,下面是我们调用 Unix `fork` 函数时会如何检查错误:

```
1      if ((pid = fork()) < 0) {
2          fprintf(stderr, "fork error: %s\n", strerror(errno));
3          exit(0);
4      }
```

`strerror` 函数返回一个文本串,描述了和某个 `errno` 值相关联的错误。通过定义下面的错误报告函数,我们能够在某种程度上简化这个代码:

```
1  void unix_error(char *msg) /* Unix-style error */
2  {
3      fprintf(stderr, "%s: %s\n", msg, strerror(errno));
4      exit(0);
5  }
```

给定这个函数,我们对 `fork` 的调用从 4 行缩减到 2 行:

```
1      if ((pid = fork()) < 0)
2          unix_error("fork error");
```

通过使用错误处理包装函数,我们可以更进一步地简化代码,Stevens 在[110]中首先提出了这种方法。对于一个给定的基本函数 `foo`,我们定义一个具有相同参数的包装函数 `Foo`,但是第一个字母大写了。包装函数调用基本函数,检查错误,如果有任何问题就终止。比如,下面是 `fork` 函数的错误处理包装函数:

```
1  pid_t Fork(void)
2  {
3      pid_t pid;
4
5      if ((pid = fork()) < 0)
6          unix_error("Fork error");
7      return pid;
8  }
```

给定这个包装函数,我们对 `fork` 的调用就缩减为 1 行:

```
1 pid = Fork();
```

我们将在本书剩余的部分中都使用错误处理包装函数。它们能够保持代码示例简洁，而又不会给你错误的假象，认为允许忽略错误检查。注意，当在本书中谈到系统级函数时，我们总是用它们的小写字母的基本名字来引用它们，而不是用它们大写的包装函数名来引用。

关于 Unix 错误处理以及本书中使用的错误处理包装函数的讨论，请参见附录 A。包装函数定义在一个叫做 csapp.c 的文件中，它们的原型定义在一个叫做 csapp.h 的头文件中；可以从 CS:APP 网站上在线地得到这些代码。

8.4 进程控制

Unix 提供了大量从 C 程序中操作进程的系统调用。这一节将描述这些重要的函数，并举例说明如何使用它们。

8.4.1 获取进程 ID

每个进程都有一个唯一的正数(非零)进程 ID(PID)。getpid 函数返回调用进程的 PID。getppid 函数返回它的父进程的 PID(创建调用进程的进程)。

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

返回：调用者或其父进程的 PID。

getpid 和 getppid 函数返回一个类型为 pid_t 的整数值，在 Linux 系统上它在 types.h 中被定义为 int。

8.4.2 创建和终止进程

从程序员的角度，我们可以认为进程总是处于下面三种状态之一：

- 运行。进程要么在 CPU 上执行，要么在等待被执行且最终会被内核调度。
- 停止。进程的执行被挂起(suspended)，且不会被调度。当收到 SIGSTOP、SIGTSTP、SIGTTIN 或者 SIGTTOU 信号时，进程就停止，并且保持停止直到它收到一个 SIGCONT 信号，在这个时刻，进程再次开始运行。(信号是一种软件中断的形式，将在 8.5 节中详细描述。)
- 终止。进程永远地停止了。进程会因为三种原因终止：1) 收到一个信号，该信号的默认行为是终止进程，2) 从主程序返回，3) 调用 exit 函数。

```
#include <stdlib.h>

void exit(int status);
```

该函数不返回。

exit 函数以 status 退出状态来终止进程(另一种设置退出状态的方法是从主程序中返回一个整数值)。

父进程通过调用 fork 函数创建一个新的运行的子进程。